The l3pdffield module
Commands to create form fields
LaTeX PDF management testphase bundle

The LaTeX Project*

Version 0.95c, released 2021-03-17

1 l3pdffield Introduction

The implementation of form fields in hyperref has some bugs, see for example https://github.com/latex3/hyperref/issues/94. This package is a first step towards the goal to review and improve the code of form fields.

Like the pdfmanagement-testphase package itself it is a temporary package: the definite home of the code is not yet decided.

It handles for now *only* checkboxes, other form fields like pushbutton, radio buttons or text fields will follow later. As currently no fonts are used, it doesn't relies on the hyperref code to initialize the form, but it can be used with hyperref.

The code requires the new PDF management. The code makes use of l3pdfxform to create the Xobjects. This code doesn't support yet the the dvips backend.

The code targets PDF 2.0. This doesn't mean that it won't work in older PDF versions, but it tries to implement requirements needed or recommended for 2.0; most importantly appearances are used by default everywhere and it deprecates `/NeedAppearances`.

Please keep in mind

- Not every PDF viewer supports form fields or all types and features.
- The handling can depend on settings in the PDF viewer. In adobe reader for example I had to disable an option to avoid that it tries to create an appearance itself
- Standards like pdf/A disable form fields too (as you typically can't change the PDF).

If hyperref is loaded before the package will suppress the deprecated `/NeedAppearances` setting. If hyperref is loaded later you should do it in the `\Form` options.

So a typical use together with hyperref could look like this

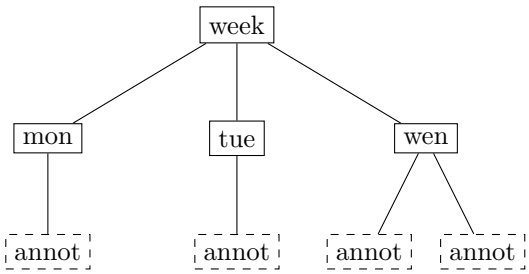
```
\RequirePackage{pdfmanagement-testphase}
\DeclareDocumentMetadata{uncompress}
\documentclass{article}
\usepackage{hyperref}
```

*E-mail: latex-team@latex-project.org

```
\usepackage{l3pdffield-testphase}
\begin{document}
\Form
```

2 Some background

A document can contain a arbitrary number of fields which can be organized in trees. The leaf fields in such a tree, the *terminal fields*, typically have widget annotations as kids which are then the actual, visual instances of the field, and allow to interact with the field. I will call such a tree a *fieldset*, nodes *fields* and the widget annotation *field annotations*.



A simple example would look like this

In many cases a fieldset consists of only one field along with its field annotation(s), but larger sets can be needed to build more complex interactions with javascript code. For example a datepicker can be built as a fieldset with various fields to represent the month and year choice and to select days.

Fields in a fieldset should have a name, for example `wen` or `week` in the example above. This name is the *partial name* of the field, the *full name* is than built from it by adding the names of the parents separated by periods. In the example above the partial name is `mon` and the full name `week.mon`. Partial names shouldn't contain periods. If two fields have the same name they will work in unison: if you enter text in one field, the text appears also in the other, such fields must have the same type and the same value and default value entry. If a field has no name it is considered to be a simple widget annotation and so only another representation of its parent.

All terminal fields should also have a type, e.g. `Btn` for a button field, or `Tx` for a textfield. The type can be set for the parent and then inherited. The fields in a fieldset can have different types.

2.1 Appearances

An field annotation may define as many as three separate appearances: the normal appearance (required), the rollover appearance and the down appearance.

An appearance can be a simple form XObjects, but in some cases the annotation can have different *appearance states*: a checkbox for example can be checked or unchecked, in this case the appearances are dictionaries which maps state names like `/Yes` and `/Off` to form XObjects.

Such form XObjects are small pictures stored in the PDF which can be referenced in various part of the PDF. They can be created with the commands of the l3pdfxform package.

The annotations cover a rectangular area on the page and the form XObjects are squeezed into this rectangle. So for the best result both should have the same ratio of width and height. Simple plain backgrounds can also be created in large size and

reused for various annotations. Form XObjects used as appearances can not be rotated, if needed one has to create a new appearance.

In PDF 2.0 widget annotations must have at least one appearance (unless the size of the annotation is zero) and the keys "*C, IC, Border, BS, BE, BM, CA, ca, H, DA, Q, DS, LE, LL, LLE, and Sy shall be ignored*". But it is quite unclear if PDF Viewer honor this, and if this make sense e.g. for text fields which require a DA entry. It is also not clear how appearances and the entries of the MK dictionary are related in a form field.

3 Commands

`\pdffield_field:nn` `\pdffield_field:nn{`⟨*key val list*⟩`}{`⟨*name*⟩`}`

This creates a new field. ⟨*name*⟩ will be used to create and reference the needed objects but it is not the direct object name, so `pdf_object_ref:n` can not be used to access (and there will not clash with object names). It is recommended to start the name with a module prefix to avoid name clashes, so e.g. `mymodule/field/1` or `mymodule/field/week`.

The list of allowed keys is described below. Typically the ⟨*key val list*⟩ should at least set the name `T`, fields that are kids in a fieldset must set the `Parent` key, this should point to a field declared before.

The command is meant as a basic command to build more complex variants like checkbox or textfields. For this reason it doesn't check if the combination of values and flags are sensible, and it uses as key names the names from the PDF reference. If you create a button field (Btn) and set MaxLen (which is only known for text fields), it will not complain.

Root fields (fields without Parent) are added automatically to the Catalog/AcroForm dictionary with

```
\pdfmanagement_add:nnx{Catalog/AcroForm}{Fields}{<obj ref>}
```

3.1 Field Keys

The command accepts the following keys.

/(PDF) key	value	required	inheritable	remark
/T	string	mostly		
/TU	string			
/TM	string			
/Ft	name	terminal fields	yes	
/setFf	list of flags		yes	
/unsetFf	list of flags		yes	
/V	various		yes	
/DV	various		yes	
/MaxLen	integer	with Comb	yes	only textfields
/Lock	object name			signature field
/SV	object name			signature field
/Opt	object name			buttons and choice fields
/AA/K	javascript			
/AA/F	javascript			
/AA/V	javascript			
/AA/C	javascript			

The following keys are relevant only for fields with variable text:

(PDF) key	/value	required	inheritable
DA	/string	yes	yes
Q	/0, 1 or 2		yes
DS	/string		
RV	/string or stream		

`T` (*field key*) This sets the partial name of the field. It shouldn't contain a period, be not empty and sensibly consist of simple ascii chars. It is normally required, see above. The value is passed through `\pdf_string_from_unicode:nnN`.

`TU` (*field key*) This sets an alternative name for user interaction. Unlike the name field it can use unicode. The value is passed through `\pdf_string_from_unicode:nnN`

`TM` (*field key*) This sets an alternative name for the export. The value is passed through `\pdf_string_from_unicode:nnN`

`Ft` (*field key*) This sets the type of the field, the value should be one of `Btn` (button), `Tx` (text), `Ch` (choice), `Sig` (signature). The value is of relevance only for terminal fields, but it can be set in a parent and then inherited.

`setFf` (*field key*) `unsetFf` (*field key*) This key accepts a list of flag names and then sets or unsets them, the resulting value is then used with the `/Ff` key. Depending on the field type some flags must be set or unset, other are optional or are ignored. The full list of flags is described later. The flag name can be given in PDF spelling (`RadiosInUnison`), in lowercase (`radiosinunison`), and as number. The list of flags are: `ReadOnly`, `Required`, `NoExport`, `Multiline`, `Password`, `NoToggleToOff`, `Radio`, `Pushbotton`, `Combo`, `Edit`, `Sort`, `FileSelect`, `MultiSelect`, `DoNotSpellCheck`, `DoNotScroll`, `Comb`, `RadiosInUnison`, `RichText`, `CommitOnSelChange`.

`V` (*field key*) This sets the value of the field. Its format varies depending on the field type, so typically commands for the various type will have to preprocess and sanitize it. The value given here is x-expanded and then added to the dictionary! See the descriptions of individual field types for further information. (Pushbutton for example don't have a value).

`DV` (*field key*) The default value, to which the field reverts when a reset-form action is executed. The format of this value is the same as that of `V`.

`MaxLen` (*field key*) Only relevant for textfields. The value is an integer and describes the maximum length of the field's text in characters. Required if the `Comb` flag is used.

`Lock` (*field key*) Only relevant for signature fields. The value is an object name which should point to a dictionary that specifies a set of form fields that shall be locked when this signature field is signed. The exact format of the dictionary is described in the PDF reference.

`SV` (*field key*) Only relevant for signature fields. The value is an object name which should point to a seed value dictionary. The exact format of the dictionary is described in the PDF reference.

`Opt` (*field key*) Only relevant for checkboxes, radiobuttons and choice fields. The value is an object name which should point to a array. The exact format of the array is described in the PDF reference.

`TI` (*field key*) Only relevant for scrollable list boxes. The value is an integer, the top index (the index in the Opt array of the first option visible in the list). Default value: 0

`I` (*field key*) For choice fields that allow multiple selection (MultiSelect flag set). The value is an object name which should point to a array. The exact format of the array is described in the PDF reference (I have no idea what exactly should be added there, perhaps some future test will make it more understandable.)

The following four keys are used to add javascript ("ECMAScript") code. The values are currently only passed through `\pdf_string_from_unicode:nnN`, but this perhaps will have to change. The keys will be ignored if a pdfstandard is used that prohibits such actions.

`AA/K` (*field key*) This adds a keystroke action to the additional action dictionary. The value is passed through `\pdf_string_from_unicode:nnN`. The action is meant for text and choice fields.

`AA/F` (*field key*) It is quite unclear if such an action make sense for non-terminal fields. This adds a format action to the additional action dictionary. The value is passed through `\pdf_string_from_unicode:nnN`. The action is meant for text and choice fields. It is quite

`AA/V` (*field key*) unclear if such an action make sense for non-terminal fields. This adds a validate action to the additional action dictionary. The value is passed through `\pdf_string_from_unicode:nnN`. It is quite unclear if such an action make sense for non-terminal fields.

`AA/C` (*field key*) This adds a calculate action to the additional action dictionary. The value is passed through `\pdf_string_from_unicode:nnN`. It is quite unclear if such an action make sense for non-terminal fields.

`DA` (*field key*) This contains instructions for the text in text fields. It is stored expanded and parentheses are added around the value.

`Q` (*field key*) The justification of the text, the allow values are 0, 1, 2 for left, centered, right.

`DS` (*field key*) these both keys are currently not implemented as it is unclear if there are of any use.

`RV` (*field key*)

`\pdffield_annot:n` `\pdffield_field:nn{`⟨*key val list*⟩`}`

This creates a new field annotation. The allowed keys are described below.

3.2 Annot keys

`width` (*annot key*) These keys allow to set the dimensions of the annotation. The value should be a command

`height` (*annot key*) that expands to a dimension expression. By default all values are zero.

`depth` (*annot key*) This sets the parent. The value should be the object name of an already declared

`Parent` (*annot key*) field.

`AP/N` (*annot key*) This keys set the normal, rollover and down appearance. The value is either a xform

`AP/R` (*annot key*) object or a dictionary mapping various states to an xform object.

`AP/D` (*annot key*) This key sets the default appearance state. The value is a name, for checkbox for

`AS` (*annot key*) example /Yes. If used it should typically have the same value as the V and DV key of the field.

`setF` (*annot key*)
`unsetF` (*annot key*)

These keys allow to set or unset the annot flags. They expect a comma lists of flag names. Allowed names `Invisible`, `Hidden`, `Print`, `NoZoom`,`NoRotate`, `NoView`, `ReadOnly`, `Locked`, `ToggleNoView`, `LockedContents`, or the lowercase variants or numbers.

`AA/F` (*annot key*)
`AA/Bl` (*annot key*)
`AA/D` (*annot key*)
`AA/U` (*annot key*)
`AA/E` (*annot key*)
`AA/X` (*annot key*)
`AA/PO` (*annot key*)
`AA/PC` (*annot key*)
`AA/PV` (*annot key*)
`AA/PI` (*annot key*)

These keys adds the `/F,` /Bl, `/D`, `/U`, `E`, `X`, `PO`, `PC`, `PV`, `PI` key to the `/AA` dictionary of the field annotation object. Their value should be javascript code. The `/AA` dictionary is suppressed if a pdf/A standard is set.

For example

```
E={app.alert('Hello');}
```

3.2.1 The widget annotation dictionary

Type Value: `/Annot`, set automatically

Subtype Value: `/Widget` this is added automatically. We use an internal dictionary which is locally copied over the one from l3pdfannot. It can be filled with keyval options.

Parent The reference to the field, automatically added.

Rect the size, calculated from the box size

Contents a text string, not really needed but an optional key should allow to set it,

AP the appearance dictionary. It should look like this `/AP <<N <<Yes 17 0 R/Off 15 0 R>>>>`. I need to test if it makes sense here to have a `/R` and `/D` entry too. The objects refer to suitable xforms.

AS should be either `/Yes` or `/Off`, and sensibly by default be the same as the V entry in the field dictionary. If they differ the AS entry wins.

A Action, this must be checked.

AA additional actions. This must be checked too.

Border, C, OC, AF, BM, Lang, P, NM, M, F, BS, H : These are not specifically needed for checkbox. An interface to add something to the used annot dictionary is needed (or already there), but probably no special key-val support for now.

MK this is what hyperref uses to set the appearance, but I explicitly leave it out and use `AP`.

Q (alignment), used by hyperref but not relevant as we don't have variable text here.

4 l3pdffield Implementation

```
⟨*package⟩
⟨@@=pdffield⟩
\NeedsTeXFormat{LaTeX2e}
\ProvidesExplPackage{l3pdffield-testphase}{2021-03-17}{0.95c}%
  {form fields}
```

4.1 hyperref specific command

hyperref sets NeedAppearances by default. As this is deprecated we disable this.

```
\csname HyField@NeedAppearancesfalse\endcsname % suppress NeedAppearances
```

4.2 local variables

```
\str_new:N \l__pdffield_tmpa_str
\cs_new_protected:Npn \__pdffield_tmpa:n #1 {}
\tl_new:N  \l__pdffield_currentfield_tl

```

4.3 messages

```
\msg_new:nnn {pdffield}{no-period}
  {
    The~field~name~`#1`~contains~a~period. \\
    This~is~not~allowed. `
  }
\msg_new:nnn {pdffield}{empty-name}
  {
    The~field~name~is~empty. \\
    This~is~not~allowed. `
  }
\msg_new:nnn {pdffield}{appearance-missing}
  {
    The~appearance~`#1`~is~missing~for~the~#2~appearance.
  }
\msg_new:nnn {pdffield}{field-keys-ignored}
  {
    The~field~`#1`~is~already~initialized\\
    The~field~keys~`#2`~are~ignored.
  }
\msg_new:nnn {pdffield}{not-implemented}
  {
    Support~for~'/#1'~is~not~implemented\\
    The~key~is~ignored.
  }
```

4.4 bitsets

A bitset for the field flag Ff and an internal copy of the annot bitset.

```
\bitset_new:Nn \l__pdffield_Ff_bitset
 {
    ReadOnly          = 1,
    Required          = 2,
    NoExport          = 3,
    Multiline         = 13,%Tx
    Password          = 14,
    NoToggleToOff     = 15,%Btn, radio button
    Radio             = 16,%Btn: Radio:    15=1, 16=0
    Pushbutton        = 17,%Btn: Checkbox: 15=0, 16=0
                           %Btn: Pushbutton: 16=1
    Combo             = 18,%Ch: Combo=1 List=0
    Edit              = 19,%Ch, Combo=1 -> + edit field
    Sort              = 20,%Ch, not relevant for view...
    FileSelect        = 21,%Tx
    MultiSelect       = 22,%Ch
    DoNotSpellCheck   = 23,%Tx, Ch (if Combo + Edit set)
    DoNotScroll       = 24,%Tx
    Comb              = 25,%Tx, requires MaxLen in dict
```

```
    RadiosInUnison    = 26,%Btn Radio
    RichText          = 26,%Tx
    CommitOnSelChange = 27,
    readonly          = 1,
    required          = 2,
    noexport          = 3,
    multiline         = 13,%Tx
    password          = 14,
    notoggletooff     = 15,%Btn, radio button
    radio             = 16,%Btn: Radio:    15=1, 16=0
    pushbutton        = 17,%Btn: Checkbox: 15=0, 16=0
                           %Btn: Pushbutton: 16=1
    combo             = 18,%Ch: Combo=1 List=0
    edit              = 19,%Ch, Combo=1 -> + edit field
    sort              = 20,%Ch, not relevant for view...
    fileselect        = 21,%Tx
    multiselect       = 22,%Ch
    donotspellcheck   = 23,%Tx, Ch (if Combo + Edit set)
    donotscroll       = 24,%Tx
    comb              = 25,%Tx, requires MaxLen in dict
    radiosinunison    = 26,%Btn Radio
    richtext          = 26,%Tx
    commitonselchange = 27
  }

\bitset_new:Nn \l__pdffield_F_bitset
  {
    Invisible      = 1,
    Hidden         = 2,
    Print          = 3,
    NoZoom         = 4,
    NoRotate       = 5,
    NoView         = 6,
    ReadOnly       = 7,
    Locked         = 8,
    ToggleNoView   = 9,
    LockedContents = 10,
    invisible      = 1,
    hidden         = 2,
    print          = 3,
    nozoom         = 4,
    norotate       = 5,
    noview         = 6,
    readonly       = 7,
    locked         = 8,
    togglenoview   = 9,
    lockedcontents = 10
  }
```

4.5 The field dictionary

The field dictionary is the main object. To be able to set values from the outside it will use a dictionary which can be filled by key-val.

```
\pdfdict_new:n   {l__pdffield/field}
\pdfdict_new:n   {l__pdffield/field/AA}
\bool_new:N \l__pdffield_root_field_bool
```

__pdffield_field:nn

__pdffield_field:nn{⟨*key val list*⟩}{⟨*obj name*⟩}

```
\cs_new_protected:Npn \__pdffield_field:nn #1 #2
  {
    \group_begin:
    \pdf_object_new:nn {__pdffield/field/#2}      {dict}
    \pdf_object_new:nn {__pdffield/field/Kids/#2} {array}
    \seq_new:c {g__pdffield_field/Kids/#2_seq}
    \tl_set:Nn\l__pdffield_currentfield_tl {#2}
    \bool_set_true:N \l__pdffield_root_field_bool
    \keys_set:nn {pdffield / field } {#1} %filter? ..., fills l__pdffield/field
    \pdfdict_put:nnx {l__pdffield/field}
      {Kids}
      {
        \pdf_object_ref:n {__pdffield/field/Kids/#2}
      }
    \pdfdict_put:nnx {l__pdffield/field}
      {Ff}
      {\bitset_to_arabic:N \l__pdffield_Ff_bitset }
    \pdfdict_if_empty:nF{l__pdffield/field/AA}
      {
        \pdfmeta_standard_verify:nT
          {annot_widget_no_AA}
          {
            \pdf_object_unnamed_write:nx {dict}{\pdfdict_use:n {l__pdffield/field/AA}}
            \pdfdict_put:nnx
              {l__pdffield/field}
              {AA}
              {\pdf_object_ref_last:}
          }
      }
    \hook_gput_code:nnn {shipout/lastpage}{pdffield} %xetex needs this ...
      { \seq_show:c{g__pdffield_field/Kids/#2_seq}
        \pdf_object_write:nx {__pdffield/field/Kids/#2}
          {
            \seq_use:cn{g__pdffield_field/Kids/#2_seq}{~}
          }
      }
    \pdf_object_write:nx {__pdffield/field/#2} { \pdfdict_use:n {l__pdffield/field} }
    \bool_if:NT \l__pdffield_root_field_bool
     {
       \pdfmanagement_add:nnx
        { Catalog / AcroForm }
        { Fields }
        {\pdf_object_ref:n {__pdffield/field/#2} }
     }
    \group_end:
  }
\cs_set_eq:NN \pdffield_field:nn \__pdffield_field:nn
```

(*End definition for* `\__pdffield_field:nn`.)

4.6 The annot dictionary

We assume that the annotation should really occupy space on the page and leave vertical mode. We also assume that keys like AP, AS are added before through keys to the dictionary. We define a dictionary for the AP content, so that we can add R and D too and the MK dictionary, so that we can setup some keys.

```
\pdfdict_new:n   {l__pdffield/annot/AA}
\pdfdict_new:n   {l__pdffield/annot/AP}
\pdfdict_new:n   {l__pdffield/annot/MK}
```

__pdffield_annot:n

```
\cs_new_protected:Npn \__pdffield_annot:n #1 % keys
  {
    \group_begin:
    \keys_set:nn {pdffield / annot } {#1}  %preset something? dimensions?
    \pdfdict_if_empty:nTF { l__pdffield/annot/AP }
      {
       \pdf_version_compare:NnT  > {1.9}
         {
           %warn about empty dict?
         }
      }
      {
        \pdf_object_unnamed_write:nx {dict}{\pdfdict_use:n{l__pdffield/annot/AP}}
        \pdfannot_dict_put:nnx {widget}{AP}{\pdf_object_ref_last:}
      }
    \pdfmeta_standard_verify:nF
      {annot_flags}
      {
        \bitset_set_true:Nn  \l__pdffield_F_bitset {Print}
        \bitset_set_false:Nn \l__pdffield_F_bitset {Hidden}
        \bitset_set_false:Nn \l__pdffield_F_bitset {Invisible}
        \bitset_set_false:Nn \l__pdffield_F_bitset {NoView}
      }
    %\pdfdict_put:nnx {l__pdffield/annot}{F}{ \bitset_to_arabic:N \l__pdffield_F_bitset }
    \pdfannot_dict_put:nnx {widget}{F}{ \bitset_to_arabic:N \l__pdffield_F_bitset }
    \pdfdict_if_empty:nF{l__pdffield/annot/AA}
      {
        \pdfmeta_standard_verify:nT
         {annot_widget_no_AA}
         {
           \pdf_object_unnamed_write:nx {dict}{\pdfdict_use:n{l__pdffield/annot/AA}}
           %\pdfdict_put:nnx
           %  {l__pdffield/annot}
           \pdfannot_dict_put:nnx {widget}
             {AA}
             {\pdf_object_ref_last:}
         }
      }
    \mode_leave_vertical:
    \hbox_to_wd:nn
      { \l__pdffield_annot_wd_dim  }
      {
        \rule [-\l__pdffield_annot_dp_dim]{0pt}{\dim_eval:n{\l__pdffield_annot_ht_dim+\l__pdf
```

```
        \pdfannot_widget_box:nnn
           { \l__pdffield_annot_wd_dim }
           { \l__pdffield_annot_ht_dim }
           { \l__pdffield_annot_dp_dim }
         \hfill
      }
    \tl_if_empty:NF \l__pdffield_currentparent_tl
      {
        \seq_gput_right:cx {g__pdffield_field/Kids/\l__pdffield_currentparent_tl _seq}{ \pdfa
      }
    \group_end:
  }
\cs_set_eq:NN \pdffield_annot:n \__pdffield_annot:n
```

(*End definition for* `\__pdffield_annot:n`.)

4.7 Appearances

TODO copy code

4.8 Annotation keys

The size of the field annotation

```
\dim_new:N \l__pdffield_annot_ht_dim
\dim_new:N \l__pdffield_annot_wd_dim
\dim_new:N \l__pdffield_annot_dp_dim

\keys_define:nn { pdffield / annot }
  {
    ,width  .dim_set:N = \l__pdffield_annot_wd_dim
    ,height .dim_set:N = \l__pdffield_annot_ht_dim
    ,depth  .dim_set:N = \l__pdffield_annot_dp_dim
    ,width  .initial:n = 0pt
    ,height .initial:n = 0pt
    ,depth  .initial:n = 0pt
  }
```

```
\keys_define:nn { pdffield / annot }
 {
   Parent .code:n =
    {
      \tl_set:Nn \l__pdffield_currentparent_tl {#1}
      \pdfannot_dict_put:nnx {widget}{Parent}{\pdf_object_ref:n{__pdffield/field/#1}}
    }
  ,AS .code:n =
    {
      \pdfannot_dict_put:nnx {widget}{AS}{#1}
    }
 }
\cs_set_protected:Npn \__pdffield_tmpa:n #1
  {
   \keys_define:nn { pdffield / annot }
     {
       AP/#1 .code:n =
```

```
        { \pdfdict_put:nnx {l__pdffield/annot/AP}{#1}{##1} }
      }
  }
\clist_map_inline:nn {N,R,D}
  { \__pdffield_tmpa:n {#1} }

\cs_set_protected:Npn \__pdffield_tmpa:n #1
 {
   \keys_define:nn { pdffield / annot }
     {
       MK/#1 .code:n =
        { \pdfdict_put:nnx {l__pdffield/annot/MK}{#1}{##1} }
     }
 }

\clist_map_inline:nn {R,BC,BG,CA,RC,AC,I,RI,IX,IF,TP}
  { \__pdffield_tmpa:n {#1} }
```

Flags.

```
\keys_define:nn { pdffield / annot }
  {
    ,setF .code:n =
      {
          \clist_map_inline:nn {#1}
           {
             \bitset_set_true:Nn \l__pdffield_F_bitset {##1}
           }
      }
    ,unsetF .code:n =
      {
          \clist_map_inline:nn {#1}
           {
             \bitset_set_false:Nn \l__pdffield_F_bitset {##1}
           }
      }
  }
```

4.9 Field keys

The names. The main name should not be empty, it is added to the dictionary when the field is created. A new name means a new field. The other names can only be set when the field is created, so we put them in the field group.

```
\keys_define:nn { pdffield /field }
  {
    ,T .code:n =
      {
        \pdf_string_from_unicode:nnN {utf8/string-raw}{#1}\l__pdffield_tmpa_str
        \str_if_in:NnT \l__pdffield_name_str {.}
          {
            \msg_error:nnx {pdffield}{no-period}{\l__pdffield_tmpa_str}
          }
        \str_if_empty:NTF\l__pdffield_tmpa_str
          {
            \pdfdict_remove:nn { l__pdffield/field }{T}
```

```
          }
          {
            \pdfdict_put:nnx { l__pdffield/field }{T}{(\l__pdffield_tmpa_str)}
          }
      }
    ,T .value_required:n = true
    ,TU .code:n =
      {
        \pdf_string_from_unicode:nnN {utf8/string}{#1}\l__pdffield_tmpa_str
        \pdfdict_put:nnx { l__pdffield/field }{TU}{\l__pdffield_tmpa_str}
      }
    ,TU .groups:n = {field}
    ,TM .code:n =
      {
        \pdf_string_from_unicode:nnN {utf8/string}{#1}\l__pdffield_tmpa_str
        \pdfdict_put:nnx { l__pdffield/field }{TM}{\l__pdffield_tmpa_str}
      }
    ,TM .groups:n = {field}
    ,Parent .code:n =
      {
        \pdfdict_put:nnx { l__pdffield/field }{Parent}
          {\pdf_object_ref:n{__pdffield/field/#1}}
        \seq_gput_right:cx {g__pdffield_field/Kids/#1_seq}
          { \exp_args:Ne \pdf_object_ref:n{__pdffield/field/\l__pdffield_currentfield_tl}}
      }
    ,Ft .choices:nn =
      { Btn, Tx, Ch, Sig }
      {
        \pdfdict_put:nnn { l__pdffield/field }{Ft}{ #1 }
      }
    ,V .code:n =
     {
       \pdfdict_put:nnx { l__pdffield/field }{V}{ #1 }
     }
    ,DV .code:n =
     {
       \pdfdict_put:nnx { l__pdffield/field }{V}{ #1 }
     }
    ,MaxLen .code:n =
     {
       \pdfdict_put:nnx { l__pdffield/field }{MaxLen}{ #1 }
     }
    ,Lock .code:n =
     {
       \pdfdict_put:nnx { l__pdffield/field }{Lock}{ \pdf_object_ref:n{#1} }
     }
    ,SV .code:n =
     {
       \pdfdict_put:nnx { l__pdffield/field }{SV}{ \pdf_object_ref:n{#1} }
     }
    ,Opt .code:n =
     {
       \pdfdict_put:nnx { l__pdffield/field }{Opt}{ \pdf_object_ref:n{#1} }
     }
```

```
    ,TI .code:n =
     {
       \pdfdict_put:nnx { l__pdffield/field }{TI}{ #1 }
     }
    ,I .code:n =
     {
       \pdfdict_put:nnx { l__pdffield/field }{I}{ \pdf_object_ref:n{#1} }
     }

  }
```

Flags. We don't add lots of individual keys but map the key names directly

```
\keys_define:nn { pdffield / field }
  {
    ,setFf .code:n =
      {
          \clist_map_inline:nn {#1}
           {
             \bitset_set_true:Nn \l__pdffield_Ff_bitset {##1}
           }
      }
    ,setFf .groups:n = {field}
    ,unsetFf .code:n =
      {
          \clist_map_inline:nn {#1}
           {
             \bitset_set_false:Nn \l__pdffield_Ff_bitset {##1}
           }
      }
    ,unsetFf .groups:n = {field}
  }

```

Keys for the AA dictionary. They all trigger javascript option.

__pdffield_setup_AAaction_key:nnn

```
%need to remove it from the checkbox ...
\cs_new_protected:Npn \__pdffield_setup_AA_key:nn #1 #2 %#1 key, #2 dict
  {
    \keys_define:nn { pdffield / #2 }
      {
         AA/#1 .code:n =
           {
             \pdf_string_from_unicode:nnN {utf8/string}{##1}\l__pdffield_tmpa_str
             \str_if_empty:NTF \l__pdffield_tmpa_str
               {
                 \pdfdict_remove:nn {l__pdffield/#2/AA}{#1}
               }
               {
                 \pdfdict_put:nnx {l__pdffield/#2/AA}
                  {#1}
                  {<</S/JavaScript/JS\l__pdffield_tmpa_str>>}
               }
           },
        AA/#1 .groups:n = {#2}
```

```
      }
  }

\__pdffield_setup_AA_key:nn {K}{field}  %keystroke
\__pdffield_setup_AA_key:nn {F}{field}  %format
\__pdffield_setup_AA_key:nn {V}{field}  %validate
\__pdffield_setup_AA_key:nn {C}{field}  %calculate
\__pdffield_setup_AA_key:nn {Fo}{annot} %{onfocus}
\__pdffield_setup_AA_key:nn {Bl}{annot} %{onblur}
\__pdffield_setup_AA_key:nn {D}{annot}  %{onmousedown}
\__pdffield_setup_AA_key:nn {U}{annot}  %{onmouseup}
\__pdffield_setup_AA_key:nn {E}{annot}  %{onenter}
\__pdffield_setup_AA_key:nn {X}{annot}  %{onexit}
\__pdffield_setup_AA_key:nn {PO}{annot} %{pageopen}
\__pdffield_setup_AA_key:nn {PC}{annot} %{pageclose}
\__pdffield_setup_AA_key:nn {PV}{annot} %{pagevisible}
\__pdffield_setup_AA_key:nn {PI}{annot} %{pageinvisible}


\keys_define:nn {pdffield/field}
  {
    DA .code:n =
      {
         \pdfdict_put:nnx{ l__pdffield_field}{DA}{(#1)}
      }
    ,Q .choices:nn = {0,1,2}
     {
       \pdfdict_put:nnn{ l__pdffield_field}{Q}{#1}
     }
    ,DS .code:n =
     {
       \msg_warning:nnn {pdffield}{not-implemented}{DS}
     }
    ,RV .code:n =
     {
       \msg_warning:nnn {pdffield}{not-implemented}{RV}
     }
  }
```

(*End definition for* `\__pdffield_setup_AAaction_key:nnn`.)

4.10 user commands

`\pdffield_setup:nn`

```
%
%\cs_new_protected:Npn \pdffield_setup:nn #1 #2
% {
%   \keys_set:nn {pdffield / #1 } {#2}
% }
⟨/package⟩
```

(*End definition for* `\pdffield_setup:nn`. *This function is documented on page* **??**.)